ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

Proof-of-Control: An Open Standard for Runtime Verifiability and Cryptographic Oversight in Autonomous AI Execution

— Prove, rather than promise, what your agents did.

Jim Schwoebel[1,2] **Ken Huang**[2,3] **Tricia Wang**[2] **Michael Casey**[2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org` [3] Cloud Security Alliance[1]

ABSTRACT

Autonomous AI agents plan multi-step workflows, invoke external APIs, modify application state, and delegate sub-tasks with minimal real-time human supervision—exposing a systemic vulnerability we call the *Verifiability Gap*: the structural impossibility of confirming that an agent operated within its authorized policy boundaries and that its controls remained active and unmanipulated. Legacy mechanisms (system prompts, static access control, embedded guardrails, post-hoc log auditing) fail against non-deterministic, path-dependent execution at machine speed. We introduce Proof-of-Control (PoC), an open standard for runtime verifiability comprising: (i) six *domains of verification* enumerating 111 auditable requirements; (ii) four *Verifiability Tiers* grading evidence by who must be trusted to believe it, with a *binary threshold* separating authenticated documentation from mechanism-generated evidence; (iii) a reference architecture coupling lifecycle interception hooks with hardware-isolated policy evaluation under IETF RATS (RFC 9334), emitting Entity Attestation Tokens over a signed Merkle execution chain; and (iv) a reproducible coverage mapping showing that eight leading frameworks (NIST AI RMF, ISO/IEC 42001, OWASP AISVS, EU AI Act, SOC 2, CSA AARM, NIST SP 800-207, MITRE ATLAS) address 27–63% of PoC requirements—almost entirely as partial matches that require the control but not operator-independent evidence of it. We state a formal policy model, an adversary model with four security properties and the assumptions each depends on, engineering design targets, and an integration roadmap across ISO/IEC JTC 1/SC 42, IETF RATS, and NIST AI 600-1. **Scope.** This is a specification and design paper: we report the standard, its formal treatment, and a descriptive gap analysis. A reference implementation and its measurement are ongoing work and are *not* claimed here; Section 9 states design targets, not results.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript; the Initiative's working groups, board, and advisory board are credited in the Acknowledgments. Additional co-authors will be added only upon explicit consent and substantive contribution.

1 Introduction

The rapid evolution from passive natural-language generation to autonomous agentic execution represents a foundational shift in computational infrastructure [1, 2]. Autonomous agents now plan multi-step operational workflows, query complex databases, invoke external APIs, modify application state, and delegate sub-tasks to secondary agents with minimal real-time human supervision. While this autonomy unlocks substantial productivity, it exposes a systemic architectural vulnerability: the *Verifiability Gap*—the absence of independent evidence of what an AI system did [3].

The Verifiability Gap describes the structural impossibility of confirming—either during execution or retrospectively—that an autonomous agent operated within its authorized policy boundaries, that mandatory security controls remained active and uncompromised, and that the agent's state transitions were free from malicious manipulation or reasoning failure. Because agents execute complex, non-deterministic actions at machine speed, human oversight and traditional log-auditing workflows cannot maintain effective operational control [4]. Existing governance implementations rely almost exclusively on operator self-attestation, static permissions, or unanchored application logs, none of which provide open, cryptographically verifiable proof of policy compliance.

Contributions and scope. This paper contributes: (1) the Proof-of-Control standard—six domains of verification, four evidence properties, four Verifiability Tiers with a binary threshold, four requirement levels aligned to the tiers, and three conformance stages—as a catalogue of 116 auditable requirements (Section 4); (2) a reference architecture integrating lifecycle interception with hardware-backed attestation under IETF RATS [5], Entity Attestation Tokens [6], and a signed hash-chained execution history in the tamper-evident-logging lineage [7, 8, 9] (Section 5); (3) a formal policy model and an adversary model with four stated security properties, the assumptions each depends on, and two attacks (snapshot substitution, split-view) that the architecture does *not* resist without the additional requirements we derive here (Sections 6–7); (4) a descriptive gap analysis mapping all 116 requirements against eight regulatory and security frameworks, with its methodological limits stated (Section 8); and (5) engineering design targets and an integration roadmap across ISO/IEC JTC 1/SC 42 [10, 11], the IETF RATS working group [12], the Agent Control Specification [13, 14], and NIST AI 600-1 [15] (Sections 9–10).

What this paper is not. We do not report an implementation or performance measurements. Proof-of-Control is a standard under public comment; a reference implementation of the enclave-resident policy engine and evidence pipeline is in progress, and its evaluation—latency distributions under realistic multi-step agent workloads, and utility cost of path-aware authorization—is deferred to a systems paper. Section 9 states the budgets such an implementation must meet; treating them as results would be unwarranted.

2 Background and Motivation

2.1 Path dependency and non-deterministic execution

Unlike legacy software executing deterministic decision trees, agents operate within LLM reasoning loops that yield non-deterministic, path-dependent trajectories. Given a single objective, an agent dynamically determines its intermediate steps, selects tools, interprets unstructured outputs, and modifies its path from real-time observations. Compliance therefore cannot be validated prior to deployment, and evaluating an individual tool call in isolation fails to mitigate path-dependent risk: an agent reading confidential records early and issuing an outbound network request later

creates a cumulative exfiltration risk that single-step inspection cannot detect. Empirical agent-security benchmarks establish both the attack surface and the weakness of per-call defenses: indirect prompt injection reliably hijacks tool-integrated agents through retrieved content [16, 17], sandboxed emulation surfaces harmful trajectories that per-step review misses [18], and dynamic agent environments show that defenses effective on isolated prompts degrade over multi-step tool-calling sequences [19, 20]. The underlying structure is not new: a sequence of individually permitted operations that together violate a policy is the classical information-flow problem [21, 22], which we return to in Section 6.1.

2.2 Why legacy governance fails

Current AI risk management relies on three primary control mechanisms, none of which resolves the Verifiability Gap.

System prompts and in-context instructions inhabit the same logical stream as untrusted user inputs, retrieved documents, and tool outputs; this co-mingling renders them vulnerable to direct and indirect prompt injection. Prompting is a probabilistic steering mechanism, not a deterministic boundary: if a model deviates from its system prompt, no execution barrier prevents the action.

Role-based access control and static identity restrict which endpoints an agent may call but operate without contextual awareness. An agent holding valid credentials for both internal database reads and external messaging retains the technical capability to exfiltrate data; static IAM cannot evaluate the context, sequence, or cumulative risk of individually authorized permissions [23].

Application-embedded guardrails run under the agent's own execution authority. For agents with code-generation or system-command tools, internal guardrails provide weak isolation—a compromised process can bypass its own application-level checks—and they generate local, unauthenticated logs that cannot be audited independently of the host operator [24].

2.3 The collapse of the post-hoc audit window

The operational window for an autonomous exploit has collapsed toward zero: indirect prompt injections embedded in public web pages or internal repositories can hijack a reasoning loop instantly [2]. Retrospective SIEM auditing is insufficient because application logs lack cryptographic immutability; an operator—or an adversary with persistence—can modify, append, or delete entries post-execution. Verification must therefore shift from reactive log auditing to inline, machine-speed cryptographic attestation anchored to hardware roots of trust [25, 26].

3 Related Work

Runtime governance for agents. The Agent Control Specification defines portable interception middleware, policy manifests, and semantic conventions for agent runtimes [13, 14], and CSA's AARM defines approve/modify/defer/deny enforcement at the action boundary [27]. These supply the *enforcement* half of agentic assurance; Proof-of-Control supplies the complementary *evidence* half—the operator-independent proof that enforcement occurred.

Remote attestation and confidential computing. The IETF RATS architecture [5, 28], Entity Attestation Tokens [6, 29], multi-verifier topologies [30], and hardware TEEs with verifiable-compute toolchains [25, 26] provide the mechanism families Proof-of-Control builds on. These prove properties of *platforms and artifacts*; Proof-of-Control extends them into a conformance regime for *agent behavior*.

Empirical agent-safety evaluation. Benchmarks for tool-integrated agents—AgentDojo [19], InjecAgent [17], ToolEmu [18], and AgentHarm [20]—establish that defenses evaluated on isolated prompts degrade over multi-step trajectories, and that harmful outcomes frequently arise from sequences of individually plausible tool calls. Proof-of-Control absorbs this as a normative requirement (path-aware authorization) rather than as a detection heuristic.

Identity substrate. Decentralized identity work—W3C DIDs [31] and the Verifiable Credentials data model [32]—together with supply-chain attestation formats [33, 34] provides the identity and provenance substrate the C1 and C5 domains build on.

Tamper-evident logging and transparency. The evidence chain is a direct descendant of Merkle authentication trees [7], history trees for tamper-evident logging [8], and Certificate Transparency [9, 35]. That literature also supplies the attacks we must resist: a log operator can equivocate (present divergent views to different auditors) or truncate, which is why CT requires gossip and consistency proofs [36, 37], and why systems such as CHAINIAC add collective signing [38]. Section 7 applies these results to agent evidence; to our knowledge, prior agent governance work does not.

Reference monitors and complete mediation. The requirement that *every* security-relevant operation be mediated by a tamper-proof, always-invoked, verifiable mechanism is the reference-monitor concept [39] and the complete-mediation principle [40]. Section 5.3 argues that agent evidence architectures satisfy the "verifiable" clause via attestation but must be designed explicitly for "always-invoked," and that a system meeting only the former produces an authenticated record that may not describe the action that actually executed.

TEE limitations. Enclave-based designs inherit a documented attack surface—architectural disclosure [41], transient execution and side-channel attacks [42]—and a vendor-rooted trust anchor. Section 7 states these as explicit assumptions rather than treating hardware attestation as trust-free.

Standards and regulation. NIST AI RMF [43] and its generative-AI profile [15], ISO/IEC 42001 [44] and 23894 [45], SOC 2 [46], the EU AI Act [47], OWASP AISVS [48], MITRE ATLAS [49], and zero-trust architecture [23] govern adjacent layers; Section **??** quantifies exactly how much of Proof-of-Control each already covers, following the coverage-mapping methodology introduced by HAARF for healthcare agent regulation [50].

4 The Proof-of-Control Standard

Proof-of-Control is *open verification* that the controls governing an agent system are implemented and hold, graded by how independently that can be verified. For each control it claims, across six domains, a conformant implementation MUST produce evidence—generated by the enforcing mechanism at execution time—that the control held; place that evidence on the Verifiability Tiers; and disclose its residual trust assumptions. A system has Proof-of-Control when, and only when, its evidence reaches Tier 3 or Tier 4 [3]. Requirements language follows RFC 2119 [51].

4.1 Six domains of verification

The standard enumerates 111 requirements across six domains, each stated as a testable *verify-that* obligation with auditor evidence guidance:

1. **Provenance** (C1): which model ran; lineage, artifact, and supply-chain origin, bound by hash-linked custody chains and signed manifests [33, 34].
2. **Privacy** (C2): what data was read and written, evidenced without re-leaking the data being protected [47].
3. **Portability** (C3): boundary crossings—organizational, jurisdictional, compute—with signed linking records so the evidence chain survives migrations.
4. **Authorization** (C4): authority granted, decisions within or against it, delegation validity; *path-aware* evaluation so a composed sequence of individually authorized calls cannot exceed the authority of its steps (Section 6.1).
5. **Identity** (C5): which agent and which principal ran, bound by short-lived delegation tokens, DIDs, and verifiable credentials [31, 32].
6. **Security** (C6): integrity of the execution environment, controls held, tools invoked, key lifecycle in hardware-backed custody.

Four cross-cutting chapters govern evidence generation and properties (binary, contemporaneous, tamper-evident, transparent), tier grading, system-surface location on the seven MAESTRO layers [52], and conformance with standardized trust-assumption disclosure.

4.2 Verifiability Tiers and the binary threshold

Evidence is graded by *who must be trusted to believe it* (Figure 1). Cryptography alone does not raise the tier; removing the trusted party does. An operator-signed log is cryptographic and still Tier 2. A vendor-rooted TEE attestation is Tier 2 unless composed with independent anchoring (e.g., a public transparency log). The *binary threshold* falls between Tier 2 and Tier 3: below it, authenticated documentation; above it, mechanism-generated evidence. The threshold makes the category procurable—"does your AI have Proof-of-Control?" is a yes-or-no question, the pattern used by every category-defining certification (FIPS 140, Common Criteria, PCI DSS, CSA STAR).

Conformance is graded on a separate axis by three named stages—Self-Declared, Third-Party Assessed, Continuously Monitored—each requiring a standardized conformance statement with declared system boundary, in-scope action classes, per-claim tiers, mechanisms, and trust-assumption disclosure. The declared inventory MUST be reconciled against automated discovery from observability streams so undeclared ("shadow") agents surface as findings.

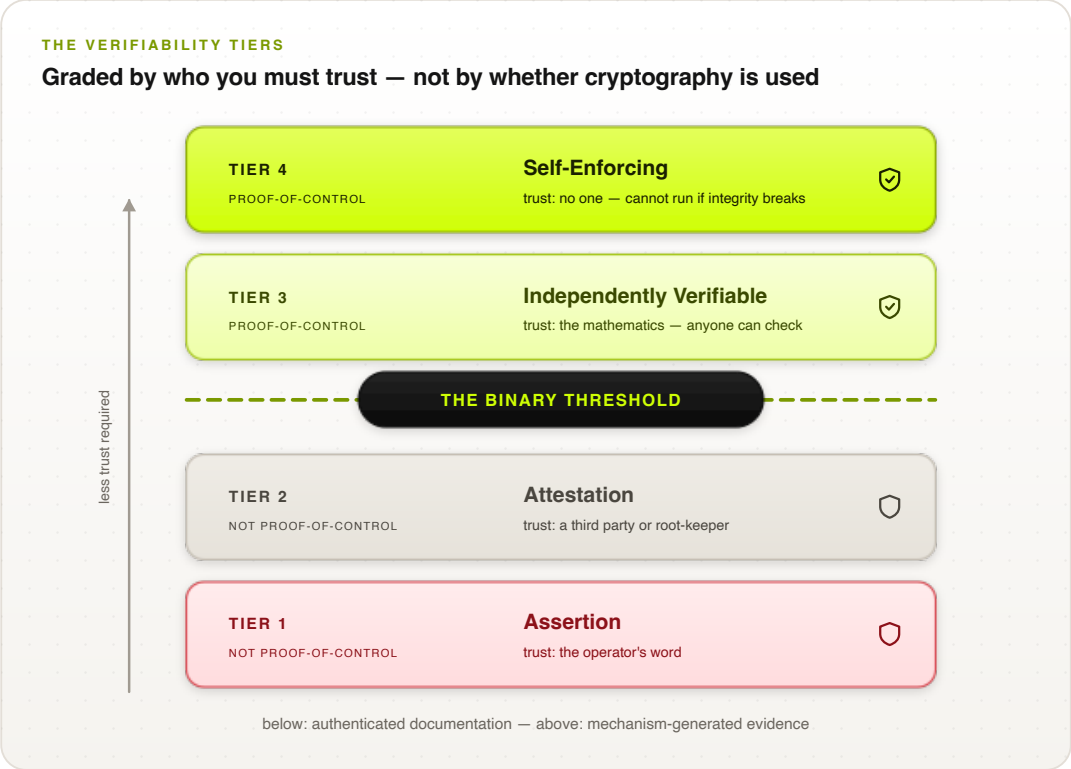

Figure 1: The four Verifiability Tiers. The binary threshold falls between Tiers 2 and 3: below it, authenticated documentation (a party must still be trusted); above it, mechanism-generated evidence. Requirement levels 1–4 align one-to-one with the tiers; meeting all Level 1–3 requirements in the claimed domains is the minimum for a Proof-of-Control claim.

5 Reference Architecture

The reference architecture integrates runtime lifecycle interception with hardware-backed attestation under the IETF RATS architecture (RFC 9334) [5, 28]. Figure 2 shows the end-to-end pipeline and its mapping onto RATS roles: every consequential action is intercepted, reduced to a canonical state snapshot, evaluated inside a hardware enclave that the operator cannot reach into, and released only after signed evidence exists; Figure 3 shows the same boundary from the evidence-generation perspective.

5.1 Lifecycle interception hooks

Proof-of-Control mandates standardized middleware hooks across the agent execution loop, leveraging the open Agent Control Specification (ACS) [13, 14]: *task initialization*, *context assembly* (RAG/memory injection), *plan generation*, *pre-call tool invocation*, *post-call tool result*, *memory write*, *sub-agent delegation* (with constraint inheritance), and *task completion*. At each point the host constructs a canonical state snapshot—agent identity, execution history, proposed action, environmental context—submitted to an isolated policy engine that returns a binding decision: ALLOW, DENY, MODIFY (inline sanitization), or ESCALATE (human-in-the-loop authorization). The gateway does not forward an action until the corresponding evidence record is durably written, and in-scope actions fail closed when evidence cannot be generated at the claimed tier.

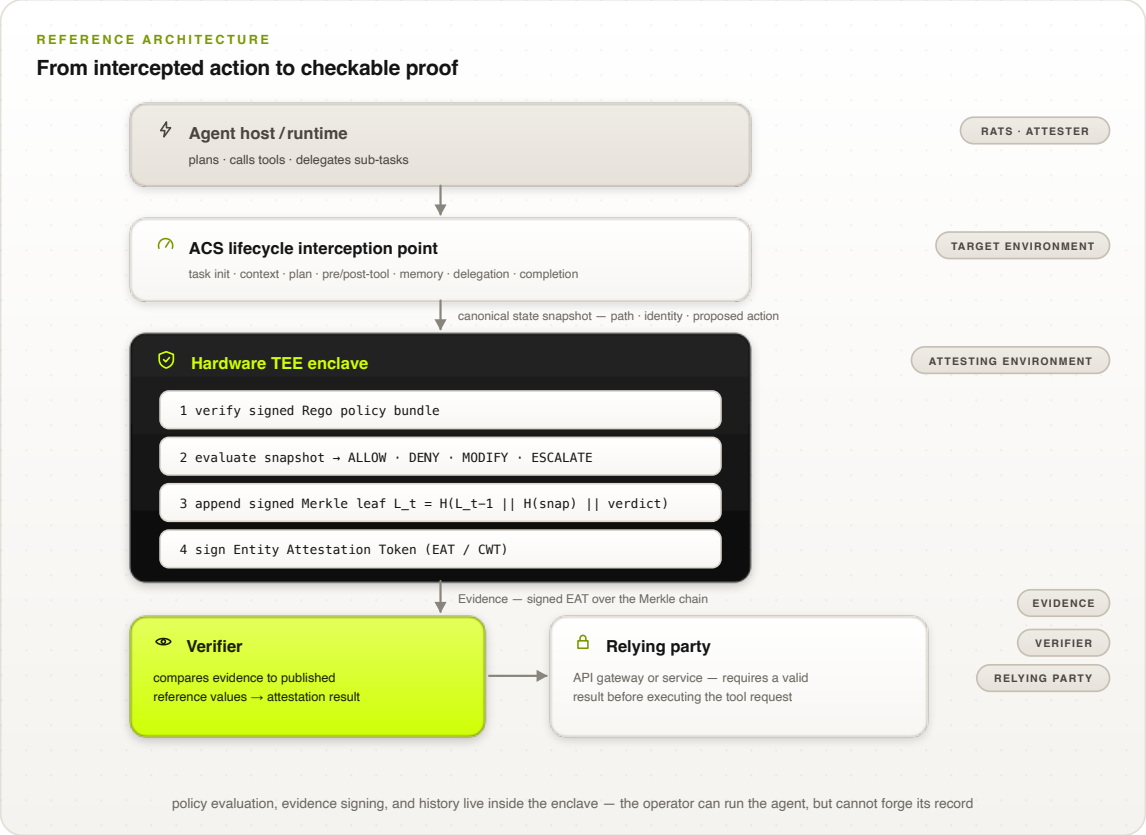

Figure 2: The Proof-of-Control reference architecture and its IETF RATS role mapping. The agent host (Attester) routes every lifecycle event through an ACS interception point; the hardware TEE enclave (Attesting Environment) verifies the policy bundle, evaluates the snapshot, extends the signed Merkle chain, and emits the Entity Attestation Token (Evidence), which a Verifier checks against published reference values before a Relying Party executes the request.

5.2 RATS role mapping

The runtime maps onto RATS roles [5]: the *Attester* is the platform housing the agent runtime, hooks, and policy engine; the *Target Environment* is the operational layer containing the runtime, model context, and proposed action; the *Attesting Environment* is a cryptographically isolated TEE (Intel TDX, AMD SEV-SNP, NVIDIA Confidential Computing) [25, 26] that measures the policy-engine binaries, verifies the policy-bundle signature, evaluates the snapshot, and signs the proof; *Evidence* is an Entity Attestation Token (CWT/JWT) carrying measurements, claims, step index, verdict, and platform signatures [6, 29]; a *Verifier* compares Evidence against reference values to produce an Attestation Result; and *Relying Parties* (API gateways, downstream services) require valid results before executing tool requests. Sub-agent delegation chains are governed by composite multi-verifier topologies [30].

5.3 Complete mediation: binding the snapshot to the effect

An enclave that evaluates a snapshot attests a precise statement: *given this snapshot, the authorized policy produced this verdict.* It does not, by itself, attest that the snapshot faithfully described the action the host subsequently performed. If the host constructs the snapshot and separately holds the tool credentials and network path, a compromised host can submit snapshot σ describing a

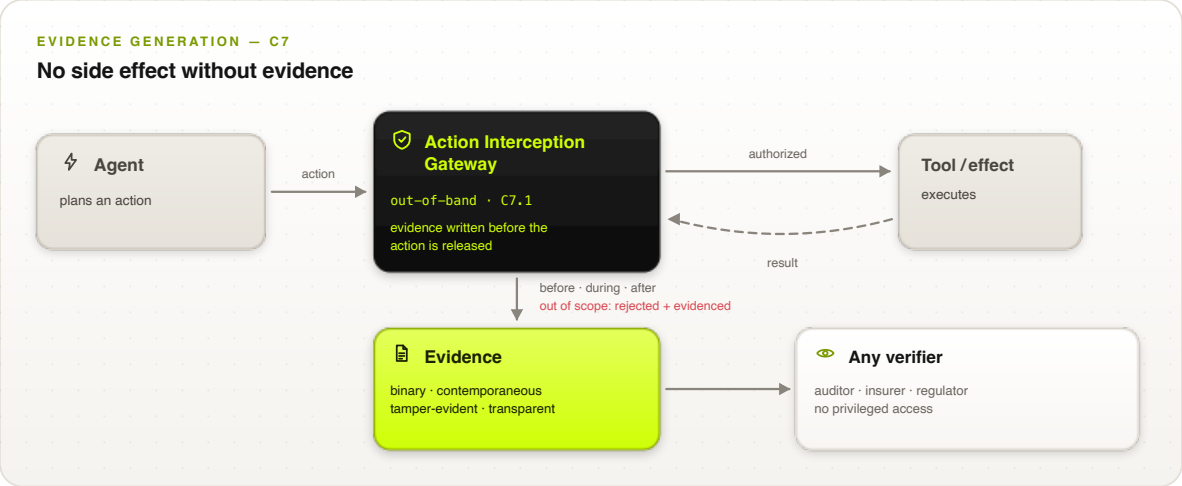

Figure 3: Evidence generation at the action boundary. The out-of-band Action Interception Gateway mediates every tool call, writes evidence before the action is released, and emits records any verifier can check without privileged access; out-of-scope actions are rejected and evidenced.

benign action, obtain an ALLOW token, and dispatch a different action $a^{\dagger}$—producing evidence that is cryptographically impeccable and materially false. We call this *snapshot substitution*; it is the agent-governance instance of the classical reference-monitor requirement that mediation be always-invoked [39, 40], and it is not closed by attestation alone.

Proof-of-Control therefore requires that the *effect channel* be mediated within the same trust boundary as policy evaluation. Concretely, at least one of the following must hold for every in-scope action class, and which one must be stated in the conformance claim:

1. **Enclave-terminated egress.** The credentials and transport used to perform the effect are held inside the Attesting Environment, which emits the request itself after evaluation; the host cannot reach the tool endpoint independently.
2. **Capability-bound dispatch.** The enclave releases a single-use, action-bound capability (a token cryptographically committing to $H(\sigma_t^{\text{snap}})$ and the target resource) that the relying party verifies before executing, so an unbound request is rejected at the far end.
3. **Attested-path mediation.** Egress is confined to a network path whose enforcement point is itself attested and configured to admit only requests carrying a valid, matching Evidence token.

Option 2 requires cooperation from relying parties and is the only one that survives a fully compromised host without enclave-terminated I/O; options 1 and 3 shrink the trusted computing base rather than eliminating host involvement. A deployment that adopts none of these may still claim Tiers 1–2, but MUST NOT claim that its evidence describes executed actions.

5.4 Merkle-chain execution history

Each intercepted step t generates a leaf

$$L_t = H(L_{t-1} \,\|\, H(\sigma_t^{\text{snap}}) \,\|\, d_t)\,, \qquad L_0 = H(\text{seed}_{\text{init}}), \tag{1}$$

where σ_t^{snap} is the canonical snapshot, d_t the verdict, and H a collision-resistant hash. The Attesting Environment signs each leaf with an enclave-bound key; per-source monotonic sequence

indices make *omission* detectable within a chain a verifier observes contiguously. Following the tamper-evident-logging literature [7, 8], the accumulated root is periodically anchored to a public transparency log or distributed trust root (asynchronously, off the hot path). As Section 7 shows, hash chaining and anchoring alone do not resist equivocation or truncation: the standard additionally requires bounded anchoring intervals and cross-verifier consistency checking, in the manner of Certificate Transparency gossip [9, 36, 37].

5.5 Canonical evidence schema

Every intercepted step yields one Evidence token: an Entity Attestation Token [6] carrying the Proof-of-Control claim set alongside the hardware attestation submodule. Listing 1 shows a representative token (JWT rendering; the CBOR/CWT encoding is byte-equivalent in claim structure), produced at the pre-call tool-invocation hook for a payment API request.

Table 1 defines each claim. Three design decisions matter. First, *payloads never leave the boundary*: the token carries `canonical_snapshot_hash`, a commitment to the full evaluated state, rather than the state itself—so a third party can later confirm exactly what was evaluated (by hashing a disclosed snapshot) without the evidence store ever holding raw data (the C2 privacy requirements). Second, *history is load-bearing*: `step_index` and `merkle_root` bind this token to every token before it, which is what makes silent omission of an inconvenient step as detectable as alteration. Third, *the policy is pinned*: `policy_bundle_hash` identifies the exact rule set that produced the verdict, so "which policy was in force?" is a claim to check, not a change-log to reconstruct.

How a verifier consumes the token. Verification is a fixed, mechanical sequence: (1) validate `attestation_signature` against the platform's attestation roots (composed with independent anchoring per the Tier-3 rule of Section 4.2); (2) compare `mr_config` to the published golden measurement of the policy engine; (3) check `policy_bundle_hash` against the signed bundle registry; (4) confirm `nonce` freshness; (5) confirm `step_index` continuity and `merkle_root` consistency with previously seen tokens for this `agent_id`; and only then (6) accept `verdict` as evidence that the control held for this action. Every step uses published materials—no privileged access to the operator's infrastructure is required, which is precisely what places this evidence above the binary threshold.

```
{
  "iss": "https://verifier.trusted-attestation.org",
  "iat": 1773532800,
  "nonce": "0x8f93e2a110c49b",
  "eat_profile": "https://standards.org/poc/v1",
  "poc_claims": {
    "agent_id": "did:web:enterprise.com:agents:finance-agent-04",
    "initiating_user": "user:auth0|84f12a9e",
    "agbom_digest": "sha256:7f8a...5f6a",
    "interception_point": "PRE_CALL_TOOL_INVOCATION",
    "step_index": 3,
    "merkle_root": "0x2c3d...2c3d",
    "policy_bundle_hash": "sha256:e3b0...b855",
    "target_resource": "api.bank.com/v1/payments",
    "canonical_snapshot_hash": "sha256:a4b3...a1b2",
    "verdict": "ALLOW"
  },
  "submods": {
    "hardware_attestation": {
      "platform": "INTEL_TDX",
      "mr_config": "0x1a2b3c4d5e6f...",
      "attestation_signature": "0x30440220..."
    }
  }
}
```


Claim	Type	Meaning
`iss` / `iat`	EAT	Token issuer and issuance timestamp.
`nonce`	EAT	Relying-party challenge; proves freshness and prevents replay of an old token for a new action.
`eat_profile`	EAT	The Proof-of-Control EAT profile (and version) this token conforms to.
`agent_id`	PoC	Decentralized identifier of the agent instance (C5); resolvable to its public keys.
`initiating_user`	PoC	The principal on whose authority the action runs—the human-to-agent binding (C5.1).
`agbom_digest`	PoC	Digest of the Agent Bill of Materials: the build, tools, and model manifest the agent runs from (C1).
`interception_point`	PoC	Which lifecycle hook produced this token (one of the eight in §5).
`step_index`	PoC	Monotonic position in the execution path; gaps expose omitted steps.
`merkle_root`	PoC	Accumulated root of the signed execution chain up to and including this step.
`policy_bundle_hash`	PoC	Digest of the signed Rego bundle that was evaluated—pins the exact policy in force.
`target_resource`	PoC	The resource the proposed action addressed (tool endpoint, table, device).
`canonical_snapshot_hash`	PoC	Commitment to the full canonical state snapshot that was evaluated; discloses nothing by itself.
`verdict`	PoC	The binding decision: ALLOW, DENY, MODIFY, or ESCALATE.
`platform` / `mr_config`	HW	TEE platform and enclave measurement; compared against published reference values to prove the authorized policy engine ran unmodified.
`attestation_signature`	HW	Hardware-rooted signature over the token by the enclave-bound key.

Table 1: The Evidence claim set. "EAT" rows are standard Entity Attestation Token claims [6]; "PoC" rows are the Proof-of-Control profile; "HW" rows form the hardware-attestation submodule.

6 Formal Model

Let $\mathcal{A}$ be the space of agent actions, $\mathcal{I}$ the space of cryptographically verifiable agent identities, and $\mathcal{S}$ the space of host state snapshots. The executed steps up to time t form the partial path $p_t = \langle a_0, a_1, \ldots, a_{t-1} \rangle \in \mathcal{A}^*$. A compliance policy is a deterministic evaluation function

$$\Pi : \mathcal{A} \times \mathcal{I} \times \mathcal{S} \times \mathcal{A}^* \longrightarrow \mathcal{D}, \qquad \mathcal{D} = \{\textsc{allow},\ \textsc{deny},\ \textsc{modify}(a'),\ \textsc{escalate}(h)\}, \tag{2}$$

where $a' \in \mathcal{A}$ is a sanitized payload and h a human-oversight routing profile. Proof-of-Control requires Π to execute inside an isolated enclave, producing per-step proof

$$\pi_t = \mathrm{Sign}_{k_{\mathrm{AE}}}(H(\sigma_t^{\mathrm{snap}}) \parallel d_t \parallel t \parallel L_t)\,, \tag{3}$$

with k_{AE} accessible exclusively to the Attesting Environment. The path argument makes the policy *path-aware*: Π evaluates the proposed action against context carried from upstream invocations, so composition attacks that are invisible to per-action evaluation are within the decision boundary. A deliberate *determinism boundary* limits claims: Proof-of-Control verifies deterministic facts of execution (which model ran, on what input, under which authority, producing which effect); it does not represent output correctness, model reasoning, fairness, or future behavior as verified. Verification is performed, not validation [44].

6.1 Relation to information-flow control

Path-aware authorization is deliberately close to, and strictly weaker than, information-flow control (IFC). The motivating hazard—an agent reads sensitive data at step i and emits at step j, with each step individually permitted—is the canonical IFC scenario, formalized as noninterference by Goguen and Meseguer [22] over Denning's lattice model [21, 53] and realized in language- and OS-level systems [54, 55, 56, 57, 58]. We inherit both the framing and the known difficulties, and we state the differences precisely rather than reinventing the field:

- **Granularity.** Classical IFC tracks labels at variable, object, or process granularity inside a trusted runtime. Proof-of-Control tracks at the granularity of *intercepted actions* at a boundary, because the agent runtime (an LLM loop with opaque internal state) is precisely what we decline to trust. We therefore obtain a coarser but externally checkable approximation.
- **Objective.** IFC systems *enforce* a flow property, often with a soundness theorem relative to a semantics. Proof-of-Control enforces a policy *and produces evidence* that the enforcement occurred, verifiable by a third party who did not observe execution. Evidence, not enforcement, is the contribution; the enforcement is an IFC-style monitor.
- **Guarantee.** We claim no noninterference result. An agent whose permitted actions are themselves harmful, or which leaks through channels outside the intercepted set (timing, content of permitted egress), is outside the claim—consistent with the determinism boundary.
- **Inherited problems.** Label creep and the declassification problem [56] predict directly the utility cost we flag as unmeasured (Section 9): a monitor that accumulates restrictions along a path will eventually refuse legitimate work, and the practical question is where declassification points belong. IFC's history suggests this, not cryptography, is the binding constraint on deployment.

6.2 Path state and evaluation complexity

Evaluating Π against the full path p_t is not implementable as written: $|p_t|$ grows without bound, and a per-step cost linear in t violates the latency target of Section 9. Proof-of-Control therefore requires policies to be expressible over a *bounded path summary*: a state ϕ_t with $|\phi_t| \leq B$ for a fixed B, updated by a fold

$$\phi_t = \mathsf{upd}(\phi_{t-1}, a_{t-1}, d_{t-1}), \qquad \phi_0 = \phi_{\text{init}}, \tag{4}$$

so that $\Pi(a, \iota, s, p_t) = \tilde{\Pi}(a, \iota, s, \phi_t)$ for the implemented $\tilde{\Pi}$. The summary is the IFC analogue of a label: it carries the accumulated sensitivity, authority, and provenance facts the policy needs (for example, "a record classified c has been read", "n tool calls remain within budget"), and nothing else.

Two consequences follow. *(i) Cost.* Per-step evaluation is $O(|\tilde{\Pi}| + B)$, independent of path length, and the snapshot committed in evidence includes $H(\phi_t)$ so a verifier can confirm the policy was evaluated against the correct accumulated state. *(ii) Expressiveness limit.* Policies requiring unbounded history—"never contact an endpoint not seen in the first k steps" for unbounded k, or exact multiset equality over arbitrary-length paths—are outside the implementable fragment and must be approximated. This is a real restriction, and it is the price of a fixed per-step budget; we state it rather than leaving path-awareness unbounded and unimplementable.

7 Adversary Model and Security Analysis

7.1 Adversary model

We consider a probabilistic polynomial-time adversary $\mathcal{A}$ that may: (i) fully control the host operating system, container runtime, orchestration layer, and network, including the ability to start, stop, reorder, replay, and drop messages; (ii) act as the *operator* of the deployment, with the motive to appear compliant—this is the distinguishing case for evidence systems, where the party producing the record is the party under scrutiny; (iii) supply arbitrary untrusted content into the agent's context (indirect prompt injection [16, 17]); and (iv) compose otherwise permitted skills and tool calls into sequences of its choosing.

We assume: **(A1)** the signature scheme and hash function are existentially unforgeable and collision-resistant; **(B1)** the Attesting Environment preserves confidentiality of its signing key and integrity of its measured code—an assumption known to be imperfect in practice under architectural and transient-execution attacks [41, 42]; **(C1)** the platform attestation root (chip vendor or its delegate) does not issue forged endorsements—a *party* assumption, which is exactly why the standard caps vendor-rooted evidence at Tier 2 absent independent anchoring; and **(D1)** verifiers can obtain published reference values and, for the properties below, communicate with at least one other verifier or monitor.

7.2 Security properties

We state four properties an evidence system should satisfy, each relative to the assumptions above.

P1 (Verdict integrity). For each token, the verdict was produced by the authorized policy bundle evaluating the committed snapshot. *Holds under (A1),(B1),(C1)*: the enclave measurement pins the policy engine, the bundle hash pins the rules, and the signature binds the verdict to $H(\sigma_t^{\text{snap}})$.

P2 (Execution fidelity). The evidence describes the action that actually executed. *Does not hold from attestation alone*: it requires complete mediation of the effect channel (Section 5.3). Without one of the three mediation options, $\mathcal{A}$ mounts snapshot substitution and P2 fails while P1 still holds—an authenticated, false record.

P3 (History integrity: append-only, no omission). No prior step can be altered, reordered, or silently dropped from a chain a verifier observes. *Holds within an observed prefix* under (A1) by hash chaining and monotonic indices; *fails against truncation* at the head, since a verifier who has seen nothing later cannot distinguish "no further steps" from "steps withheld." The standard therefore requires anchoring within a bounded interval Δ, which reduces the undetectable window to at most Δ and makes a missing anchor itself an alarm.

P4 (Non-equivocation). All relying parties see the same history. *Does not hold from signing and anchoring alone*: $\mathcal{A}$ can fork the chain and present divergent prefixes to different verifiers (split-view), a known attack on transparency systems [9, 36, 37]. Mitigation requires cross-verifier consistency checking (gossip), a witness quorum co-signing roots [38], or anchoring to a ledger with single-history consensus; the standard requires one of these for Tier-3 claims where multiple relying parties act on the same agent's evidence.

7.3 Formal treatment of P2 and P4

P1 and P3 follow directly from standard signature and hash-chain arguments. P2 and P4 are the properties round-one analysis showed do *not* follow from attestation and chaining alone, so we state them precisely.

Definition 1 (Evidenced execution)**.** *Fix an agent identity ι and a relying party R. An effect e executed at R is evidenced if there exists a token τ accepted by a verifier such that τ commits to a snapshot digest $H(\sigma^{\text{snap}})$, a target resource r, and verdict $d = \textsc{allow}$, where σ^{snap} describes exactly e and r is the endpoint at which e occurred.*

Definition 2 (Execution-fidelity experiment $\mathsf{Exp}^{\text{fid}}$)**.** *$\mathcal{A}$ controls the host, network, and operator role as in Section 7. $\mathcal{A}$ interacts arbitrarily with the Attesting Environment $\mathcal{E}$, obtaining tokens and capabilities of its choice. $\mathcal{A}$ wins if a compliant relying party R executes an effect e that is not evidenced per Definition 1.*

Theorem 1 (Execution fidelity under capability-bound dispatch)**.** *Let $\mathcal{E}$ release, for each $\textsc{allow}$ verdict, a single-use capability $c = \text{Sign}_{k_{\mathcal{E}}}(H(\sigma^{\text{snap}}) \,\|\, r \,\|\, \text{nonce})$, and let compliant R execute a request only if it presents a c that (i) verifies under an enclave key whose measurement matches a published reference value, (ii) names R's own resource r, (iii) carries an unused nonce, and (iv) matches the request's canonical form against $H(\sigma^{\text{snap}})$. Then for any PPT $\mathcal{A}$,*

$$\Pr[\mathcal{A} \textit{ wins } \mathsf{Exp}^{\text{fid}}] \;\le\; \mathbf{Adv}^{\text{EUF-CMA}}_{\text{Sig}}(\mathcal{A}) \;+\; \mathbf{Adv}^{\text{coll}}_{H}(\mathcal{A}),$$

under assumptions (A1), (B1), (C1).

Proof sketch. Suppose $\mathcal{A}$ wins: R executed e with no token evidencing e at r. By R's check (i), the presented c verified under $k_{\mathcal{E}}$, which by (B1) never leaves $\mathcal{E}$ and by (C1) is bound to the attested measurement; so either $\mathcal{A}$ forged a signature—bounding this case by $\mathbf{Adv}^{\text{EUF-CMA}}$—or c was genuinely issued by $\mathcal{E}$. In the latter case, by check (iv) the executed request's canonical form hashes to the $H(\sigma^{\text{snap}})$ inside c. If σ^{snap} did not describe e, then two distinct canonical forms share a digest, bounding this case by $\mathbf{Adv}^{\text{coll}}_{H}$. Otherwise σ^{snap} describes e; by check (ii)

the resource matches, and $\mathcal{E}$ issues c only on an ALLOW verdict whose token it also emits—so e is evidenced, contradicting the win condition. Check (iii) prevents replay of a capability against a second effect. □

Proposition 1 (Necessity of mediation)**.** *Without a check binding the executed request to $H(\sigma^{\text{snap}})$ at the point of effect,* $\Pr[\mathcal{A}$ *wins* $\mathsf{Exp}^{\text{fid}}] = 1$ *for an adversary controlling the host.*

Proof. $\mathcal{A}$ submits σ describing benign e', obtains an ALLOW token, and dispatches $e \neq e'$ using host-held credentials. No verifier check is violated: the token is genuine and $\mathcal{E}$'s statement ("policy allows e'") is true. e is unevidenced, so $\mathcal{A}$ wins with certainty. □

Proposition 1 is why C7.1.4 is a requirement rather than a recommendation: an evidence architecture without effect mediation admits a total break of P2 while every signature verifies.

Definition 3 (Equivocation)**.** *$\mathcal{A}$ equivocates if two verifiers V_1, V_2 accept chains Λ_1, Λ_2 for the same ι such that neither is a prefix of the other.*

Theorem 2 (Equivocation is detectable and attributable)**.** *Let each chain root at index i be signed by $\mathcal{E}$, and let V_1 and V_2 exchange (root, index) pairs—by direct gossip, via a witness quorum of which at least one member is honest, or through a single-history ledger. If $\mathcal{A}$ equivocates, then under (A1) and (D1) the honest parties jointly hold two validly signed roots at a common index with distinct values, which constitutes a non-repudiable proof of equivocation attributable to $k_{\mathcal{E}}$; detection probability is* 1*.*

Proof sketch. Non-prefix chains differ at some least index i. Both V_1 and V_2 hold signed roots at i; exchanging them under (D1) reveals distinct values. Forging either root reduces to signature forgery, excluded by (A1). The pair is self-contained evidence, so detection does not depend on trusting either verifier's account. □

Theorem 2 clarifies the requirement's shape: gossip does not *prevent* equivocation, it makes it detectable and attributable—the same guarantee Certificate Transparency provides [9, 37], and the reason C7.3.3 accepts any of three mechanisms that establish a shared view. Without such a mechanism, the split-view attack succeeds silently.

Table 2 maps representative threats to the property they attack and the requirement that addresses them.

7.4 Constructing Tier 4

Tier 4 requires that verification gate operation: the system cannot perform in-scope actions unless integrity holds. This is a construction, not an assertion, and it follows directly from mediation option 2 of Section 5.3. Let the relying party accept a request only if it carries a capability c_t committing to $H(\sigma_t^{\text{snap}})$, the target resource, and a fresh nonce, signed by the enclave key whose measurement matches a published reference value. Then:

- if the enclave is not running, or its measurement does not match, no valid c_t can be produced, and every in-scope request is refused at the relying party—the system *fails closed by construction* rather than by operator policy;
- the halting decision is therefore *not under operator control*, which is the property Tier 4 claims and which an in-process "halt" flag cannot provide, since a compromised host can patch it out;

Threat	Attacks	Mitigation and residual risk
Operator suppresses enforcement or edits records	P1, P3	Policy logic and signing keys are enclave-isolated; altering the bundle changes the measurement and verifiers reject. Residual: (B1),(C1).
Snapshot substitution (dispatch $\neq$ evaluated action)	P2	Complete mediation of the effect channel (Section 5.3). Residual: options 1 and 3 still trust host non-interference with transport configuration.
Log truncation at the head	P3	Bounded anchoring interval Δ; missing anchors alarm. Residual: actions within the final Δ window.
Split-view / equivocation	P4	Gossip, witness co-signing, or consensus anchoring. Residual: colluding witnesses.
Indirect prompt injection	P1 (indirect)	The enclave evaluates deterministic rules independently of model reasoning, so an injected instruction cannot itself authorize an out-of-scope action. Residual: in-scope harmful actions—a safety, not verifiability, question [16, 18].
Skill-composition attacks	P1	Path-aware evaluation: Π takes the partial path as an argument (Section 6), so composed escalation is inside the decision boundary. Residual: utility cost, unquantified (Sections 6.1, 9).
Cascading sub-agent exploitation	P1–P4	Delegation hooks enforce constraint inheritance; sub-agent evidence verified via multi-verifier topologies [30].
Enclave compromise	P1–P4	Out of scope by (B1); disclosed as a trust assumption and bounded by re-attestation and key rotation.

Table 2: Threats mapped to the security property attacked and the mitigating requirement. Properties P2 and P4 are *not* obtained from attestation and hash chaining alone—the analysis that motivates the additional requirements in Sections 5.3 and 7.

- the residual dependency is the relying party's enforcement and the attestation root (C1), both of which must appear in the trust-assumption disclosure.

This construction also clarifies the vendor-root tension: a TEE attestation alone remains Tier 2, because (C1) is a party assumption. A deployment reaches Tier 3 by composing it with independent anchoring so that a verifier need not rely solely on the vendor's word about which code ran, and Tier 4 by additionally making valid evidence a precondition of execution. The tiers therefore grade *trust structure*, not mechanism labels—consistent with the standard's own rule that cryptography is a mechanism, not a tier.

7.5 Deployability of capability-bound dispatch

Theorem 1 places a real burden on the ecosystem: relying parties must verify capabilities. This is the strongest objection to the architecture, and plausibly why it does not already exist. Three observations bound the difficulty.

The mechanism is not new. Action-bound capabilities are the classical capability model [59, 60] in modern dress, and macaroons already provide exactly the needed shape—bearer tokens with cryptographic caveats, verifiable by the target service without contacting the issuer [61], deployed over ordinary HTTP headers. A relying-party check is a signature verification plus a caveat comparison, not new infrastructure.

Adoption is incremental, and the first steps need no third party. Mediation options 1 and 3 (Section 5.3) are unilateral: an operator can terminate egress inside the enclave, or route it through an attested proxy, with no cooperation from anyone. Within an organization, the API gateway is already a relying party under the same administrative control, so capability checking can be enabled

there first. Cross-organizational enforcement follows contractually (as SOC 2 evidence requirements already propagate through vendor agreements), and only then by standardization—which is the purpose of the EAT profile work in Appendix A.

The residual gap is honest. Endpoints that will never verify capabilities—arbitrary third-party SaaS, legacy systems—can be covered only by options 1 or 3, whose guarantee is weaker: they shrink the trusted computing base rather than removing host involvement, and a deployment must disclose which option covers which action class. We do not claim a universal solution; we claim a specified property, three implementations of it with different residual trust, and a disclosure requirement that makes the difference legible to a relying party.

8 Gap Analysis: What Existing Frameworks Already Require

This section characterizes where Proof-of-Control sits relative to existing frameworks. We describe it as a *gap analysis* rather than an evaluation: it measures coverage of our own requirement set, so it cannot by construction validate that set. Its purpose is descriptive—to locate the layer Proof-of-Control occupies—and its limitations are stated below.

8.1 Method

Following the coding methodology of HAARF [50], every Proof-of-Control requirement was coded against eight frameworks as Exact Match (equivalent clause in scope and intent), Partial Match (same topic, but without operator-independent evidence or at lesser depth), or No Match. Coverage $= (\text{EM} + \text{PM})/N$ over the $N = 116$ requirements. Coding is at section granularity with per-row rationales and clause attributions; the sheet, rubric, and computation script are public, so any reader can re-derive the numbers or re-code a framework.

8.2 Results

Figure 4 and Table 3 report the mapping. Two patterns are consistent. First, the partial-match band is wide and the exact-match band thin: NIST AI RMF attains the highest coverage with zero exact matches, because governance frameworks require nearly every control Proof-of-Control verifies while never requiring mechanism-generated evidence that the control held. Second, no-match concentrates in the evidence-property, tier-grading, and disclosure chapters: no coded framework grades evidence by how independently it can be verified, requires trust-assumption disclosure, or specifies self-enforcing execution.

Framework	Exact	Partial	None	Coverage
NIST AI RMF [43]	0	73	43	63%
OWASP AISVS [48]	16	55	45	61%
ISO/IEC 42001 [44]	8	62	46	60%
EU AI Act [47]	8	57	51	56%
SOC 2 [46]	8	57	51	56%
CSA AARM [27]	13	46	57	51%
Zero Trust (SP 800-207) [23]	8	44	64	45%
MITRE ATLAS [49]	0	31	85	27%

Table 3: Requirement-level coverage (single-coder seed data). The exact-match band is thin everywhere: frameworks require the *controls* but rarely operator-independent *evidence* that they held.

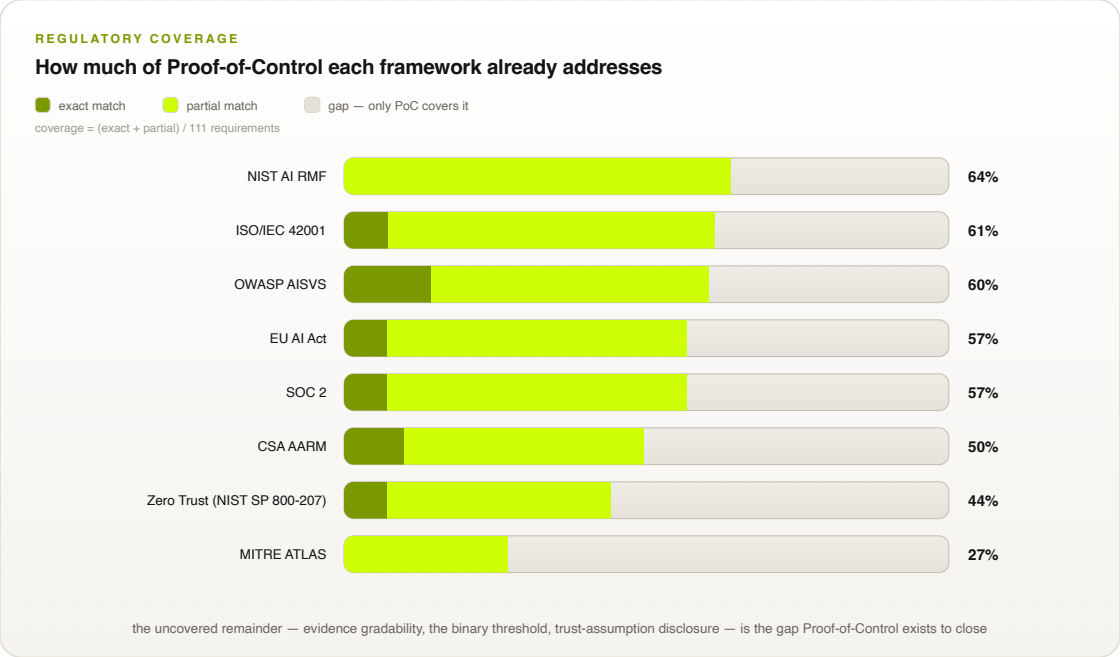

Figure 4: Coverage of the Proof-of-Control requirements by eight external frameworks (single-coder seed data; see limitations). Dark lime: exact matches; bright lime: partial matches; the remaining track is the residual gap.

8.3 Limitations of this analysis

Four limitations bound what may be concluded. **(i) Direction.** The metric asks how much of Proof-of-Control each framework covers; a novel requirement set scores incumbents low by construction, so these numbers argue that Proof-of-Control occupies a distinct layer—not that it is correct or necessary. **(ii) Single coder.** The seed coding was performed by one author, and inter-rater reliability is therefore undefined. The public rubric specifies a two-coder protocol with Cohen's κ [62] reported per framework, and adversarial re-coding by parties who did not author the requirements; that study is planned and its results will supersede Table 3. **(iii) Granularity.** Codings are assigned at section level and inherited by the requirements within a section, which overstates uniformity. **(iv) One-directional.** We do not report the reverse mapping—requirements those frameworks impose that Proof-of-Control omits (e.g., NIST AI RMF's organizational governance functions, ISO/IEC 42001's management-system obligations, SOC 2's availability and processing-integrity criteria). Proof-of-Control is deliberately narrower than any of them; a complete comparison must show both directions, and the reverse map is future work.

8.4 Comparison with runtime governance systems

Table 4 compares Proof-of-Control against the mechanisms it is most often conflated with. Because Proof-of-Control is a standard rather than a deployed system, the comparison is on *specified* properties: what each approach requires an implementation to provide. The closest active work—ACS [13, 14] and CSA AARM [27]—specifies runtime interception and enforcement but leaves the resulting record under operator control; transparency-log systems [9, 38] provide non-equivocation machinery but are not agent-aware; confidential-computing pipelines [26] attest computation but do not model agent authority or delegation. Proof-of-Control's contribution is the composition and

the grading rule, not any single mechanism.

Property	Prompts / guardrails	Static RBAC	ACS / AARM	Transparency logs	Proof-of-Control
Mediates every action	No	At API layer	Yes	N/A	Yes (required)
Path-aware decisions	No	No	Partial	N/A	Yes (required)
Record integrity	None	Operator logs	Operator-run receipts	Hash-chained, gossiped	Hash-chained + anchored
Independently checkable	No	No	With operator access	Yes	Yes (Tier 3+)
Non-equivocation	No	No	Not specified	Yes (with gossip)	Required at Tier 3+
Execution fidelity (P2)	No	No	Not specified	N/A	Required (mediation)
Trust disclosure	No	No	No	Implicit	Standardized

Table 4: Specified properties of Proof-of-Control against adjacent approaches. Entries describe what each approach *requires*; Proof-of-Control is a specification, and no implementation comparison is claimed.

9 Design Targets and Open Engineering Questions

Because agent workflows involve synchronous tool invocations, governance must operate at machine speed. We state design targets that a conformant implementation should meet; **these are budgets, not measurements**, and validating them is the purpose of the reference implementation now in progress.

- **Interception overhead:** a target of < 15 ms added latency per intercepted step, decomposed into enclave transition, snapshot canonicalization, policy evaluation, signing, and chain append. Each term is measurable and each is contested in the literature; enclave transition and canonicalization are the terms we expect to dominate.
- **Anchoring interval Δ:** the undetectable truncation window (Section 7) is bounded by Δ, so Δ trades verifiability against anchoring cost. The standard requires Δ be declared; Table 5 gives working guidance derived from the quantity actually at risk—the volume of in-scope action that can be suppressed within one window—rather than from convention.

- **Path-aware authorization utility cost:** evaluating against carried context can reject workflows that per-action evaluation would admit. The false-rejection rate on realistic benign workloads is *unmeasured*, and a monitor that blocks legitimate work will not be deployed. Quantifying this—on agent benchmarks such as [19, 18]—is required future work and may bound how aggressive path-awareness can be.
- **Privacy-mechanism obligations:** where C2 admits a zero-knowledge path, an implementation must state the relation proven (e.g., "the accessed field set is a subset of the consented set for purpose p"), the setup assumption, and the leakage profile of the disclosed statement. "Uses ZKPs" is not a conformance claim; the tier assigned depends on setup provenance, since a single-party trusted setup reintroduces exactly the party-trust the threshold forbids.

Action class	Exposure per window	Suggested Δ	Rationale
Irreversible external effect (payment, trade, medication order)	Unbounded financial or safety harm	$\leq$ 1 s, or per-action anchoring	Truncation must not be able to hide a completed irreversible act; per-action anchoring collapses the window to zero at the cost of one round trip.
Reversible external effect (ticket, message, provisioning)	Bounded, recoverable	$\leq$ 1 min	Recovery is possible if detection is prompt; anchoring cost amortizes over many actions.
Read-only access to regulated data	Disclosure already occurred	$\leq$ 5 min	The act cannot be undone, but detection latency drives breach-notification timelines rather than loss magnitude.
Internal, non-effecting steps (planning, retrieval)	Low direct harm; evidential value	$\leq$ 1 h	Value is reconstructive; batching is appropriate.

Table 5: Working guidance for the anchoring interval Δ. Δ bounds the window in which head truncation is undetectable (Section 7), so it should be chosen from the harm that one window's worth of suppressed actions can cause. These are proposals for working-group ratification, not measured results.

Implementation strategies we expect to be necessary—policy engines compiled to WebAssembly or native code executing inside the enclave to avoid IPC per step, hardware-accelerated Ed25519/ECDSA P-256 signing, and asynchronous anchoring off the hot path—are stated as engineering guidance, not results. The verification-bandwidth economics motivating the budget are structural [4]: as execution cost falls toward zero, verification becomes the scarce factor, so checking must be cheap, binary, and mechanical.

10 Discussion

Standards-integration targets (ISO/IEC JTC 1/SC 42, IETF RATS, ACS, NIST AI 600-1) and the phased implementation roadmap are given in Appendices A and B; this section states what the work does not yet establish.

10.1 Limitations and status

Six limitations are stated deliberately. **(1) No implementation or measurement.** This paper specifies and analyzes; it does not evaluate a running system. The design targets of Section 9 are unvalidated, and the reference implementation is ongoing work. **(2) Determinism boundary.** Proof-of-Control verifies execution facts, not output correctness, fairness, or intent; validation remains a human responsibility, and an in-scope harmful action is a safety question the standard explicitly does not claim to solve. **(3) Trust is reduced, not eliminated.** Tier 3–4 claims rest on assumptions (A1),(B1),(C1) of Section 7; TEE compromise and vendor-root failure are out of scope by assumption, and enclave attack literature [41, 42] shows these assumptions are imperfect. The standard's response is mandatory disclosure, not a claim of trustlessness. **(4) Gap analysis is single-coder and one-directional** (Section 8.3); the multi-coder study with κ and the reverse mapping are pending. **(5) Binary-threshold formalization.** The Tier-2/Tier-3 line is under dedicated cryptographic and complexity-theoretic review, including impossibility results bounding what agentic verification can achieve efficiently; a formal characterization may move the line. **(6) Availability coupling.** Tier-4 fail-closed operation makes availability depend on the

evidence path; deployments must assess and disclose this, and the construction of Section 7.4 deliberately places the halt outside operator control, which is a safety property and an operational risk simultaneously.

Open working-group questions—identity/authorization ownership, verifiable-but-unlinkable identity, evidence continuity across jurisdictions as a possible fifth evidence property, and continuous-monitoring cadence—are tracked publicly in the specification's open-issues register.

10.2 Document status

Proof-of-Control is a working draft under public comment, developed openly with a public specification repository, audit checklist, coding sheets, and figure tooling. This paper documents the standard and its formal treatment; a systems paper reporting the reference implementation and its measurement is the intended companion. We invite adversarial review of the binary threshold, the mediation requirement, and the coverage coding in particular.

11 Conclusion

The claims-based world, in which relying parties accept an operator's word for what an autonomous system did, does not scale to agents no one can watch. Proof-of-Control replaces attestation with mechanism-generated, hardware-anchored, independently checkable evidence: six domains of verifiable facts, a binary threshold that makes verifiability procurable, an architecture that intercepts every consequential action, and a coverage evaluation demonstrating that no existing framework occupies this layer. The specification, audit checklist, coding sheets, and reference tooling are developed openly under the Advanced AI Society and are freely licensed for implementation.

Acknowledgments

This work is developed within the Proof-of-Control Initiative of the Advanced AI Society. The authors thank the domain working groups and contributors whose input shaped the draft standard, including Bob Blessing-Hartley, Advait Patel, David Thomson, Drummond Reed, and Hart Montgomery (Linux Foundation Decentralized Trust), and the Cloud Security Alliance MAESTRO and AARM communities. The Proof-of-Control Lab is being established as a community lab at Linux Foundation Decentralized Trust.

A Standards Integration

ISO/IEC JTC 1/SC 42: Proof-of-Control provides a technical realization for active SC 42 workstreams [10, 11, 63]—a technical annex for auditable runtime controls under ISO/IEC 42001 Clauses 8–9 (supplying cryptographically verifiable compliance evidence during certification audits) [44], operationalization of ISO/IEC 23894 risk treatment as executable, hardware-verified policy bundles [45], and an SC 42/SC 27 bridge coupling AI oversight with confidential-computing standards [64]. *IETF RATS:* an Internet-Draft establishing an AI-agent EAT profile [6, 29]—standardizing claims for lifecycle hook types, policy-bundle hashes, Agent Bill of Materials (AgBOM) digests, and step indices—and an extension of multi-verifier attestation to sub-agent delegation chains [30]. *ACS:* Proof-of-Control supplies the missing cryptographic layer for ACS [13, 14], anchoring its decision engine in TEEs and signing decision tokens. *NIST AI 600-1:* Proof-of-Control provides concrete implementations for the generative-AI profile's measurement controls [15,

65, 66]—enforcing that deployment configurations cannot bypass security controls and satisfying continuous security verification with attestation tokens for every tool call and state transition.

B Implementation Roadmap

Figure 5 summarizes the path to ratification and adoption. **Phase 1 (months 1–6): open specification and reference core**—core schema extensions on the ACS repository, open-source Rust reference enclave policy engines, and the IETF Internet-Draft for the AI-agent EAT profile. **Phase 2 (months 7–12): multi-vendor interoperability and SDO balloting**—SC 42 working groups, conformance testing across Python, Node.js, and Rust agent frameworks, and telemetry integration with OpenTelemetry and OCSF. **Phase 3 (months 13–24): ratification and deployment**—normative annexes in ISO/IEC 42001 and 23894, RFC publication, and deployments across financial, healthcare, and public-sector workloads. On the adopter side, the same three phases carry an organization from foundational key, signing, and logging infrastructure through expanded proof coverage to continuously monitored operation—each phase reaching readiness for one conformance stage.

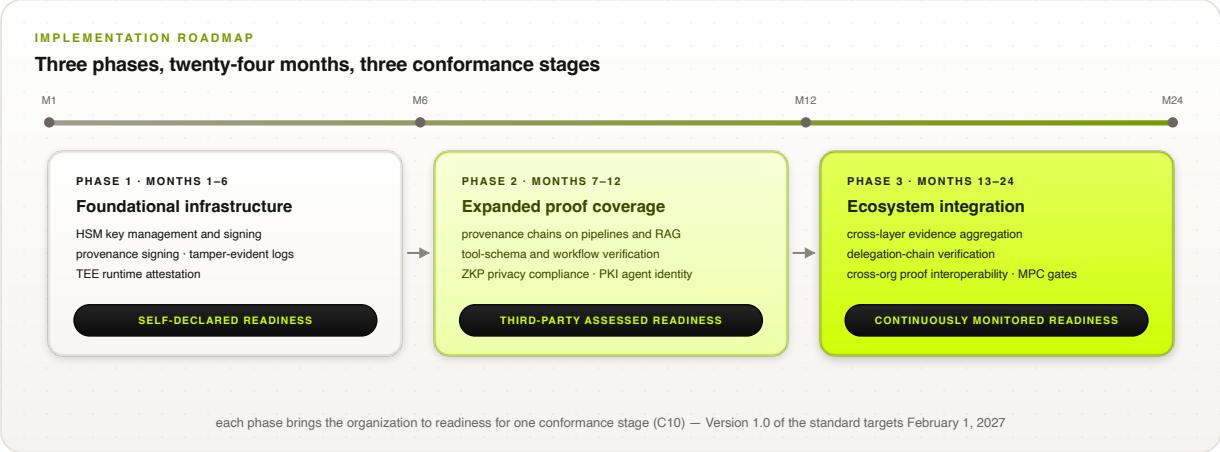

Figure 5: Implementation roadmap. Each phase brings an adopting organization to readiness for one conformance stage; on the standards track, Version 1.0 targets February 1, 2027.

References

[1] Palo Alto Networks. A complete guide to agentic AI governance. `https://www.paloaltonetworks.com/cyberpedia/what-is-agentic-ai-governance`, 2026.

[2] Airia. What is AI agent governance? A framework for keeping agents in check. `https://airia.com/blog/what-is-ai-agent-governance-a-framework-for-keeping-agents-in-check/`, 2026.

[3] Advanced AI Society. Open verification: the proof-of-control standard for agents, working draft v0.1.4. `https://advancedaisociety.org/`, 2026. Specification repository: requirement chapters C1–C10, appendices, audit checklist, and regulatory coding sheets.

[4] Christian Catalini, Xiang Hui, and Jane Wu. Some simple economics of AGI. *arXiv preprint arXiv:2602.20946*, 2026. `https://arxiv.org/abs/2602.20946`.

[5] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023.

[6] Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, Internet Engineering Task Force, 2025.

[7] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378. Springer, 1988.

[8] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*, 2009.

[9] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013.

[10] ISO/IEC JTC 1. SC 42: Artificial intelligence — subcommittee overview. `https://jtc1info.org/sd-2-history/jtc1-subcommittees/sc-42/`, 2026.

[11] Nemko Digital. ISO/IEC JTC 1/SC 42: Leading AI standards for 2025. https://digital.nemko.com/standards/iso-iec-jtc-1sc-42, 2025.

[12] IETF RATS Working Group. Remote ATtestation procedureS (rats): Charter and documents. https://datatracker.ietf.org/group/rats/about/, 2026.

[13] Agent Control Standard. Agent control standard launches open framework for runtime governance of AI agents. https://www.businesswire.com/news/home/20260527326259/en/Agent-Control-Standard-Launches-Open-Framework-for-Runtime-Governance-of-AI-Agents, 2026.

[14] Microsoft Command Line. Agent control specification: Portable runtime governance for AI agents. https://commandline.microsoft.com/agent-control-specification-runtime-governance/, 2026.

[15] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. NIST AI 600-1, NIST, 2024.

[16] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[17] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[18] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024.

[19] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[20] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[21] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[22] Joseph A. Goguen and José Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, 1982.

[23] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero trust architecture. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[24] NETBankAudit. Generative AI controls and risk assessments for financial institutions. https://www.netbankaudit.com/resources/generative-ai-controls-risk-assessments, 2026.

[25] Red Hat. Attestation in confidential computing. https://www.redhat.com/en/blog/attestation-confidential-computing, 2026.

[26] EQTY Lab. Verifiable compute: AI integrity suite. https://www.eqtylab.io/blog/verifiable-compute-press-release, 2026.

[27] Cloud Security Alliance. Autonomous action runtime management (AARM). https://cloudsecurityalliance.org/, 2026. Contributed by Vanta.

[28] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures architecture. Internet-Draft draft-ietf-rats-architecture-07, Internet Engineering Task Force, 2021.

[29] Ned Smith. TCG attestation framework and IETF remote attestation procedures. https://trustedcomputinggroup.org/wp-content/uploads/2_N.Smith_TCG-JRF02292024.pdf, 2024. Trusted Computing Group.

[30] IETF RATS Working Group. Remote attestation with multiple verifiers. Internet-Draft draft-ietf-rats-multi-verifier-00, Internet Engineering Task Force, 2026.

[31] World Wide Web Consortium. Decentralized identifiers (DIDs) v1.0. https://www.w3.org/TR/did-core/, 2022.

[32] World Wide Web Consortium. Verifiable credentials data model v2.0. https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[33] OpenSSF. SLSA: Supply-chain levels for software artifacts. https://slsa.dev/, 2026.

[34] Coalition for Content Provenance and Authenticity. C2PA technical specification. https://c2pa.org/, 2026.

[35] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, Internet Engineering Task Force, 2021.

[36] Linus Nordberg, Daniel Kahn Gillmor, and Tom Ritter. Gossiping in CT. Internet-Draft draft-ietf-trans-gossip-05, Internet Engineering Task Force, 2018.

[37] Melissa Chase and Sarah Meiklejohn. Transparency overlays and applications. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[38] Kirill Nikitin, Eleftherios Kokoris Køgias, Philipp Jovanovic, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Justin Cappos, and Bryan Ford. CHAINIAC: Proactive software-update transparency via collectively signed skipchains and verified builds. In *26th USENIX Security Symposium*, 2017.

[39] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, ESD/AFSC, Hanscom AFB, 1972.

[40] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[41] Victor Costan and Srinivas Devadas. Intel SGX explained. Cryptology ePrint Archive, Report 2016/086, 2016.

[42] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *27th USENIX Security Symposium*, 2018.

[43] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, NIST, 2023.

[44] International Organization for Standardization. ISO/IEC 42001:2023 — information technology — artificial intelligence — management system. `https://www.iso.org/standard/81230.html`, 2023.

[45] International Organization for Standardization. ISO/IEC 23894:2023 — artificial intelligence — guidance on risk management. `https://www.iso.org/standard/77304.html`, 2023.

[46] AICPA. Trust services criteria (2017, with 2022 revised points of focus). `https://www.aicpa-cima.com/resources/download/trust-services-criteria`, 2022.

[47] European Union. Regulation (EU) 2024/1689 — artificial intelligence act. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj`, 2024. Official Journal of the European Union, 12 July 2024.

[48] OWASP Foundation. Artificial intelligence security verification standard (AISVS), v1.0. `https://github.com/OWASP/AISVS`, 2026.

[49] MITRE Corporation. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. `https://atlas.mitre.org/`, 2026.

[50] Task Force for AI Agents in Healthcare. HAARF: A unified, lifecycle-centric regulatory framework for autonomous AI agents in medical environments. `https://github.com/Task-force-for-AI-agents-in-Healthcare/haarf`, 2026. Coverage-mapping methodology (EM/PM/NM rubric, coding sheets, reproducible coverage computation).

[51] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, Internet Engineering Task Force, 1997.

[52] Ken Huang. MAESTRO framework: Navigating risks in multi-agent AI systems. `https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro`, 2025. Cloud Security Alliance.

[53] Dorothy E. Denning and Peter J. Denning. Certification of programs for secure information flow. *Communications of the ACM*, 20(7):504–513, 1977.

[54] Andrew C. Myers and Barbara Liskov. A decentralized model for information flow control. In *16th ACM Symposium on Operating Systems Principles (SOSP)*, 1997.

[55] Andrew C. Myers. JFlow: Practical mostly-static information flow control. In *26th ACM Symposium on Principles of Programming Languages (POPL)*, 1999.

[56] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[57] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. Making information flow explicit in HiStar. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.

[58] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. Information flow control for standard OS abstractions. In *21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.

[59] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[60] Henry M. Levy. *Capability-Based Computer Systems*. Digital Press, 1984.

[61] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.

[62] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[63] LexAI Europe. ISO/IEC standards on AI. `https://www.lexai-europe.com/1901/standardisation/iso-iec-ai`, 2025.

[64] International Electrotechnical Commission. Coordinated standardization for AI and data security in the era of generative AI: Strategic proposals. `https://iec.ch/system/files/2025-10/wsd_2025_combinedpdf.pdf`, 2025.

[65] CipherNorth. NIST AI 600-1 and AI RMF: Managing risk in generative AI. `https://www.ciphernorth.com/blog/nist-ai-risk-management-framework-rmf`, 2026.

[66] Modulos. Implement NIST AI risk management framework. `https://www.modulos.ai/nist-ai-rmf/`, 2026.